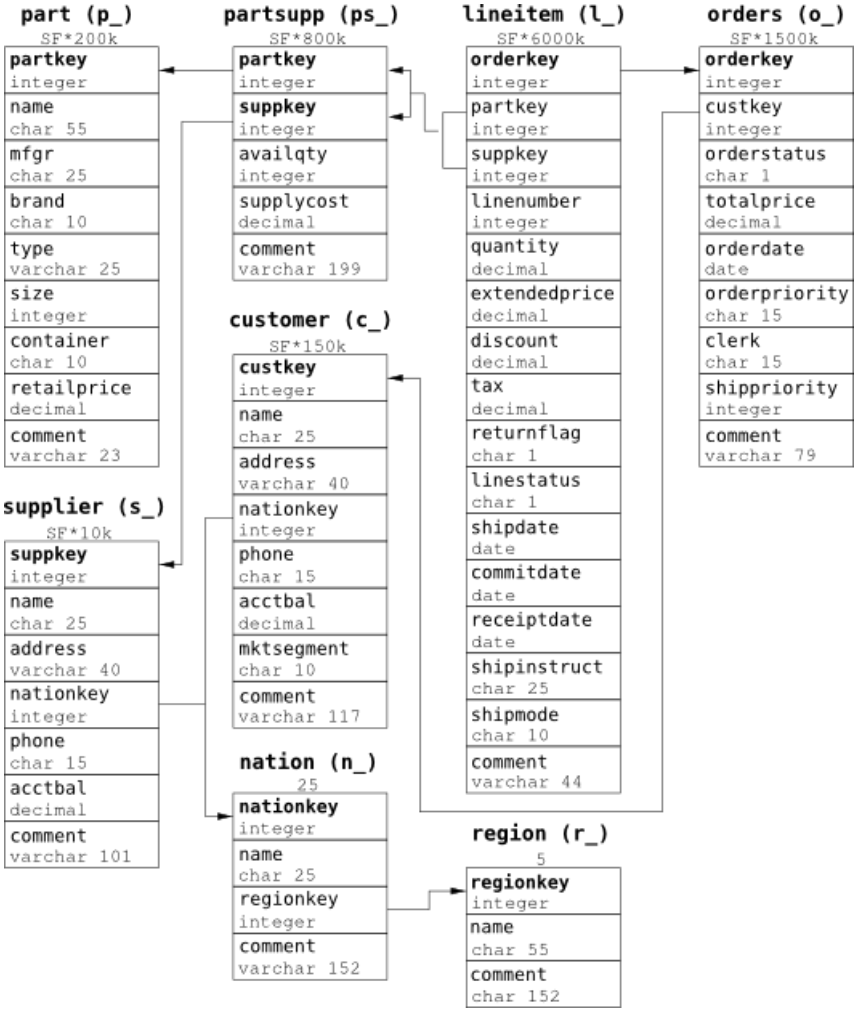
TPC-H, Profiling & mmap

FDE Tutorial Sessions, Week 3

TPC-H & GNU Tools

TPC-H

prefix (partkey → p_parkey)
SF = 1



time

Runs a command and reports how long it took (on **stderr**)

```
$ time sort big.txt > out.txt

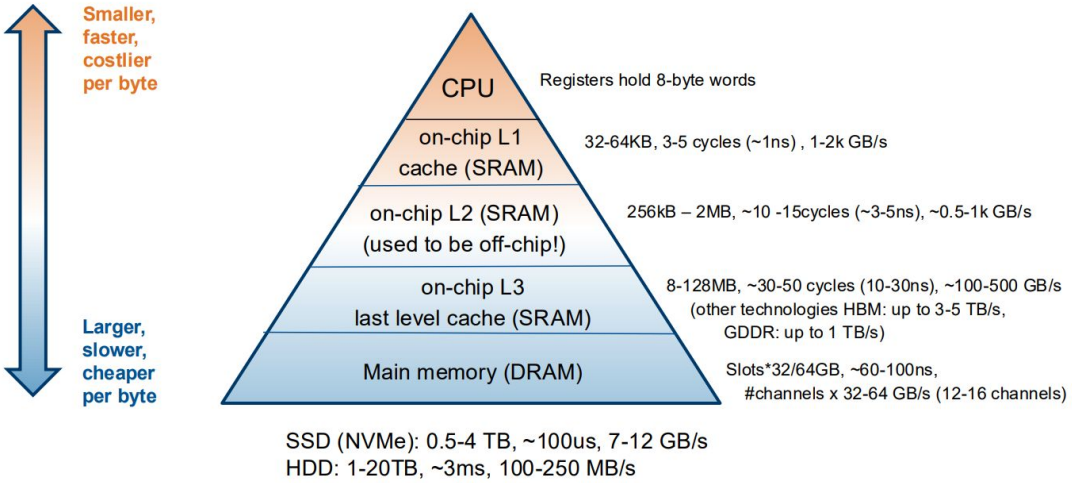

real    0m3.214s
user    0m5.902s
sys     0m0.611s
```

- **real** - wall-clock time, start to end
- **user** - CPU time spent in your code, **summed over all processes and threads** (can be > real)
- **sys** - CPU time spent in the kernel (system calls such as read, write)

Src: `man time`

Attendance sheet

Memory hierarchy



- Higher levels: small and fast
- Lower levels: big and slow
- Memory moves in **cache lines**
- LLC = last-level cache, the last stop before main memory

Image src: lecture notes; Src: U. Drepper, What every programmer should know about memory

perf stat

```
$ perf stat -e cycles,instructions,\
  LLC-loads ./prog args

2,400,000,000  cycles
3,600,000,000  instructions
   40,000,000  LLC-loads
```

- **IPC** = instructions / cycles. Low IPC → the CPU is mostly **waiting**, often for memory
- Divide counters by the **amount of work** (per row, per pair) to compare runs of different size
- **LLC-load** = reads from last-level cache (usually L3)

Src: https://perf.wiki.kernel.org/index.php/Tutorial; https://www.brendangregg.com/perf.html

perf record / perf report – sampling

```
$ perf record -g ./prog args   # writes perf.data
$ perf report                  # interactive view
```

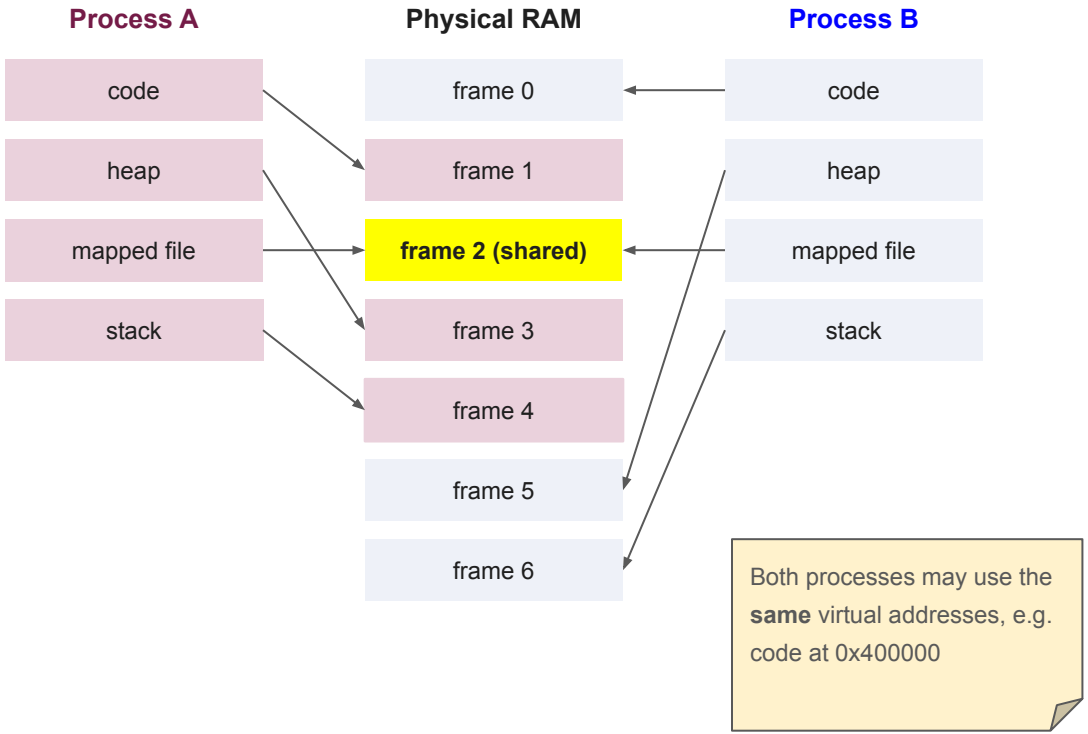
- Interrupts the program thousands of times per second and notes **which instruction** runs
- Build with **RelWithDebInfo!**

Src: https://perf.wiki.kernel.org/index.php/Tutorial; man perf-record

mmap

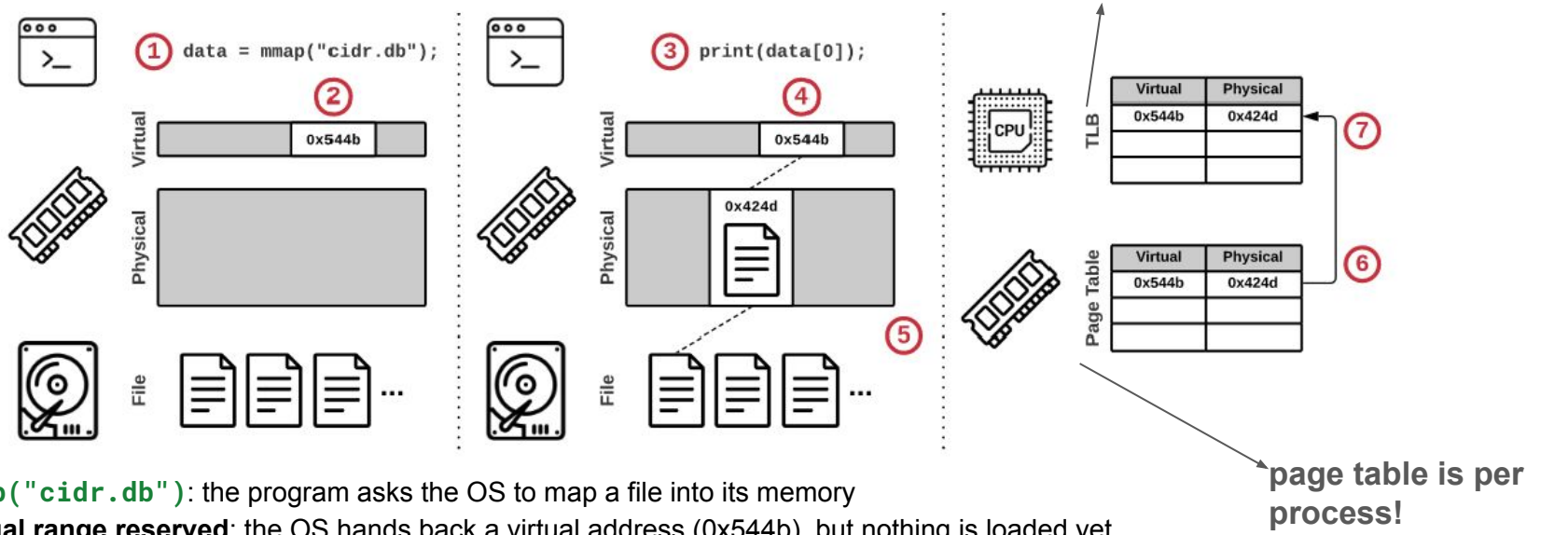
Virtual vs physical addresses

- **Physical address** = where a byte really sits in the RAM chips
- **Virtual address** = what your program sees (every pointer is one)
- Each process gets its **own virtual address space** → the same address in two processes means different bytes
- The CPU's **MMU** (=*memory management unit*) translates **virtual** → **physical** on every access, one page at a time



Src: https://en.wikipedia.org/wiki/Virtual_memory; OSTEP, ch. 18 Paging; https://www.heuristic42.com/blog/62/rethinking-writing-files-with-memory-mapping-and-c/

mmap



1. `mmap("cidr.db")`: the program asks the OS to map a file into its memory
2. **Virtual range reserved**: the OS hands back a virtual address (0x544b), but nothing is loaded yet
3. `print(data[0])`: the program reads from that address for the first time
4. **Address lookup fails**: the CPU finds no physical page behind 0x544b -> ***page fault***
5. **File loaded from disk**: the kernel reads that part of the file into a free physical page (0x424d)
6. **Page table updated**: the kernel records that *virtual 0x544b* now maps to *physical 0x424d*
7. **TLB cached**: the CPU caches that mapping so later accesses are fast

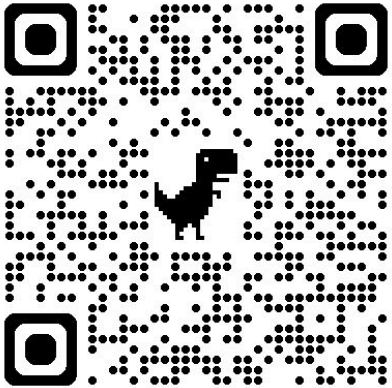
TLB - used by the CPU to work with addresses fast; in case an address is not found, it is looked up in the process page table, then stored in the TLB (for some time)

Src: Lecture slides

For questions: veneta.kireva@tum.de

Thank you!

Link to these slides:



Next week: threads, SQL;
building on this week's code - make sure to be familiar with it!